ASSIGNMENT-4

NAME: ARAVIND HTnO: 2303A51215

BATCH:04 DATE:09-02-26

Objective

To design, develop, and deploy a simple ERC20 Token smart contract using Solidity, following Ethereum standards.

Requirements

• Install VS Code

• Install Solidity extension in VS Code

• Use Remix Ethereum IDE (online) or Hardhat (optional)

• Basic understanding of blockchain and Ethereum

• MetaMask wallet (for deployment testing)

Practical Implementation

Step 1: Development Environment Setup

• Install VS Code

Step 2: Create ERC20 Smart Contract

Create a Solidity smart contract that:

• Defines token name, symbol, decimals, and total supply

• Allows token transfer between accounts

• Maintains balances using mappings

• Emits events for transparency

Step 3: Solidity Code (ERC20Token.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
/*
Simple ERC20 Token Implementation
*/
contract MyERC20Token {
```

```
// Token details
string public name = "MyToken";
string public symbol = "MTK";
uint8 public decimals = 18;
uint public totalSupply;
// Mapping to store balances
mapping(address => uint) public
balanceOf;
// Event to log transfers
event Transfer(address indexed from,
address indexed to, uint value);
// Constructor runs once during
deployment
constructor(uint initialSupply) {
totalSupply = initialSupply * (10 **
uint(decimals));
balanceOf[msg.sender] = totalSupply;
}
// Transfer function
function transfer(address to, uint
value) public returns (bool success) {
require(balanceOf[msg.sender] >=
value, "Insufficient balance");
balanceOf[msg.sender] -= value;
balanceOf[to] += value;
emit Transfer(msg.sender, to,
value);
return true;
}
}
```

Step 4: Explanation of Logic

• *mapping(address => uint) stores token balances*

• *constructor() assigns total supply to deployer*

• *transfer() moves tokens between accounts*

• *require() ensures sender has enough balance*

• *emit Transfer() logs transactions on blockchain*

Step 5: Deployment

• *Compile the contract using Solidity Compiler*

• *Deploy using Remix VM (London) or MetaMask*

• *Provide initial supply during deployment (e.g., 1000)*

Code:

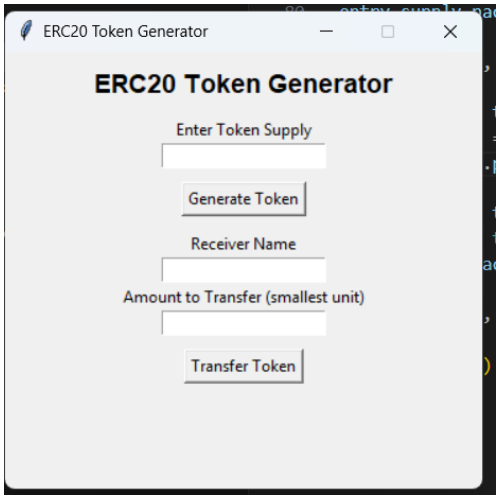
```
import tkinter as tk

from tkinter import messagebox


# ---------------- ERC20 TOKEN LOGIC ----------------

class ERC20Token:

    def __init__(self, token_number):

        self.name = "MyToken"

        self.symbol = "MTK"

        self.decimals = 0


        # Total supply in smallest units

        self.total_supply = token_number * (10 ** self.decimals)


        # Creator owns all tokens initially

        self.balances = {"creator": self.total_supply}


    def transfer(self, sender, receiver, amount):

        if amount > self.balances.get(sender, 0):

            return False, "Insufficient Balance"

        else:

            self.balances[sender] -= amount

            self.balances[receiver] = self.balances.get(receiver, 0) + amount
```

```
            return True, "Transfer Successful"

    def balance_of(self, user):
        return self.balances.get(user, 0)

# ---------- GUI FUNCTIONS ----------
def create_token():
    try:
        token_number = int(entry_supply.get())
        if token_number <= 0:
            raise ValueError

        global token
        token = ERC20Token(token_number)

        messagebox.showinfo(
            "Token Created",
            f"Token Name: {token.name}\n"
            f"Symbol: {token.symbol}\n"
            f"Total Supply: {token_number} {token.symbol}"
        )
    except ValueError:
        messagebox.showerror("Error", "Please enter a valid token supply")

def transfer_token():
    try:
        receiver = entry_receiver.get()
        amount = int(entry_amount.get())

        if receiver == "" or amount <= 0:
            raise ValueError

        success, message = token.transfer("creator", receiver, amount)

        if success:
```

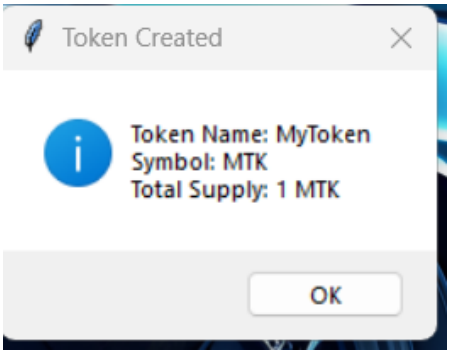
```
            messagebox.showinfo("Transfer Status", message)
        else:
            messagebox.showerror("Transfer Failed", message)

    except NameError:
        messagebox.showerror("Error", "Please generate token first")
    except ValueError:
        messagebox.showerror("Error", "Enter valid receiver and amount")

# ---------- GUI DESIGN ----------
root = tk.Tk()
root.title("ERC20 Token Generator")
root.geometry("360x330")
root.resizable(False, False)

tk.Label(root, text="ERC20 Token Generator", font=("Arial", 14, "bold")).pack(pady=10)

tk.Label(root, text="Enter Token Supply").pack()
entry_supply = tk.Entry(root)
entry_supply.pack()

tk.Button(root, text="Generate Token", command=create_token).pack(pady=10)

tk.Label(root, text="Receiver Name").pack()
entry_receiver = tk.Entry(root)
entry_receiver.pack()

tk.Label(root, text="Amount to Transfer (smallest unit)").pack()
entry_amount = tk.Entry(root)
entry_amount.pack()

tk.Button(root, text="Transfer Token", command=transfer_token).pack(pady=10)

root.mainloop()
```

output:

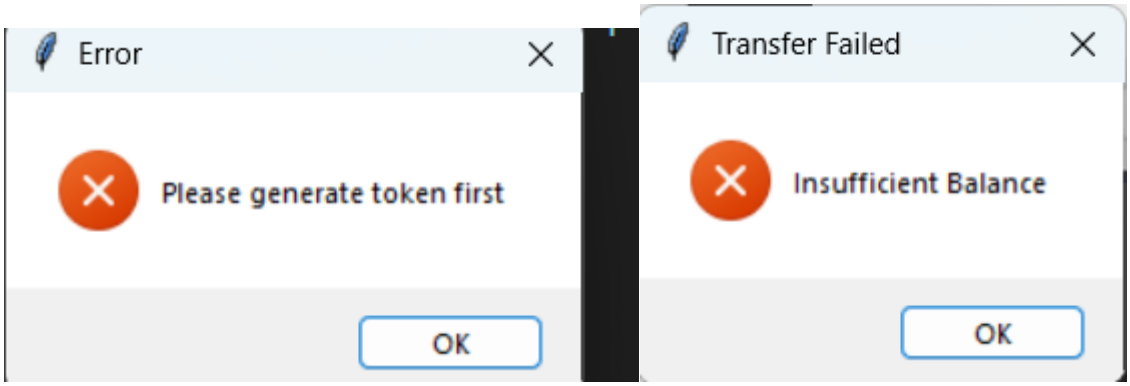
interface:



Token generation:



Raising errors:



require()/Transfer: